

An Executive Overview

What the pipeline is, how it works, and what is inside the package - August 5, 2026

Executive summary

General Pipeline v4.2 is a reusable software-engineering pipeline: a ready-to-use starter package plus a five-stage workflow that any new AI-assisted project can adopt on day one.

It was not designed in the abstract. It was distilled from a real nine-milestone customer project (EF-AI), hardened with feedback from live projects, and in v3 broadened by a cross-project harvest of six more projects ratified by a 13-seat blind review council; v3.1 (July 2026) then folded in the first field run of the pipeline itself (executable wave-close checklists, a living experience document, risk-tiered review depth); and v3.2 (July 2026) added the owner review pack and trust telemetry, shaped by an external research corpus, and recorded a long-term autonomy ladder as a north-star design - explicitly NOT activated at its higher rungs. v3.3 (July 2026) took the first measured step on that ladder: waves now close on agent reviews and mechanical checks, while the owner conducts a deep review session - his own tests, the diffs, and every commit - at each milestone, protected by an automatic fallback if the new cadence ever misses something the old one would have caught. v3.4 (July 2026) closed the last gap in the lifecycle: production bugs now flow through the same disciplined review as new features and ship in verified, documented fix packages that the owner personally validates outside any sandbox before deployment - and every fix automatically becomes a recorded lesson. v3.5 (July 2026) was cut on the first real production dataset - seven defects that slipped past every automated gate, all of them at the boundary between the system and the real world - and answers with outward-facing checks: the shipped configuration must boot, a fresh environment must start cold, a human following only the docs must succeed, and a black-box script must walk the real customer journey against the deployed product. v4.0 (July 2026) is the first MAJOR cut informed by the outside world: three independent AI market studies, cross-checked against each other, confirmed that the pipeline's four core mechanics have no equivalent among roughly twenty-five surveyed commercial and open-source players - and supplied the improvements worth absorbing. v4.0 hardens the trust boundary (review rules can only come from the protected branch, never from the change being reviewed - a defense grounded in three real 2026 exploits), adds measured test-quality reporting and cross-model review at the highest risk tier, keeps the customer-journey check running on a schedule after deployment, and - honestly - starts measuring the pipeline's own friction: every safety step skipped under pressure is now recorded, and a step skipped three times triggers a review of the step itself. v4.1 (July 2026) is the first cut driven by an audit of the pipeline itself rather than by new ideas: the review council inspected the actual files instead of the summary it was given, and found that three of the checks the pipeline had formally adopted did not work - one had never run at all, and one called a program that did not exist. They are fixed, and they now fail loudly instead of quietly reporting success, because a declared control that silently passes is worse than no control. v4.1 then closes the gap that let this happen: the pipeline's own decision records - previously prose that nothing read - are now validated data, checked by a single dependency-free script, guarded by a set of deliberately broken example records that prove the

checker still works, and enforced by the project's first true merge gate. It also records, for the first time, a list of things the pipeline has decided NOT to build and why. v4.2 (August 2026) is the cut where the audit turned on its own author. The chair filed the two measurement instruments the council had been demanding - one that records whether a control was ever skipped, and one that traces an escaped defect back to the decision that let it in - and then claimed, in the same document, that a new automated check was already installed. It was not; it was still a paragraph. All six reviewers found that sentence independently, by reading the code instead of the document. The check now exists, with deliberately broken examples proving it fires, and the false claim is recorded in the instrument as its own largest finding. The same review found five more real defects: a security check in the sister pipeline that let a pasted password through whenever the line happened to end in a comment, a credential-detection pattern that had been written down but never connected to anything for eight releases, a self-test structurally incapable of reaching the two rules it was credited with, a duplicate-record check that could never fire, and a validator that could not see the two new documents the whole review was about. All fixed, each with a test that fails when the fix is removed. The lesson the cut is named for: a control nobody has watched fail is a rumour. Every opinion in it traces back either to direct measurement or to a documented industry incident.

The value is simple: repeatable speed with built-in safety. Parallel AI agents do the work; mandatory independent reviews and a default-deny permission system keep that work safe; and every project the pipeline runs makes the next one faster, because lessons are captured as reusable rules.

What it is, in 30 seconds

The pipeline has two layers:

Layer 1 - Starter Package	Layer 2 - Workflow (five stages)
Pre-filled scaffolding - rules, permissions, decisions, automation - that a new project copies and fills in (about 121 files).	Bootstrap, then Plan, then Wave Execution, then the Per-Wave Review Duo, then Milestone Closure. Most time is spent in the middle three.

How it works

- 1. Bootstrap (once per project)** Set the project up from the starter package. An executable check (make bootstrap-check) confirms nothing critical was skipped before work begins - including, in v3, a web/API security baseline (no default-admin password, no plaintext credentials).
- 2. Milestone Plan (per milestone)** The agent drafts a plan - in v3, with a short gap analysis of what already exists versus what must be built; a human reviews and approves it before any code is written. The human is the planning authority.
- 3. Wave Execution (parallel)** Independent agents implement the approved scope in parallel, each running its own implement-test-fix loop, with automatic commit-gate checks running as they go.
- 4. Per-Wave Review (after each wave)** Two independent reviewers - a Code Reviewer and a Tester - inspect the work with fresh eyes and confirm every acceptance criterion has a passing test. They never review their own code. (In v3 the security review moved to milestone closure.)

5. Milestone Closure A blocking security review runs before any deploy; a quality gate checks evidence and test coverage (every acceptance criterion must have a citing test); lessons are captured as reusable rules; and if the milestone deploys, go-live readiness is verified.

Four principles hold it together:

- **Human approves, agent proposes.** The agent opens drafts and plans; a person signs off before waves run and before any code merges.
- **Default-deny safety.** The agent is told what it may NOT do. The most catastrophic actions (destructive git or database operations, committing secrets) are blocked outright, not just discouraged.
- **Mandatory fresh-eyes review.** Every wave ends with an independent code review and an independent tester - a second pair of eyes on every change - and a blocking security review runs at milestone closure before any deploy.
- **Capture and compound.** Decisions, session logs and lessons accumulate in the repo, so the pipeline gets measurably better with each project instead of starting from zero.

Why it is credible

Measured directly in the source project (nine milestones of observation):

- Parallel agent dispatch produced a 4-6x reduction in wall-clock time.
- Fresh-eyes review caught blocking issues at every milestone where it ran.
- Contract-lock plus grep-verification produced zero cross-agent integration drift.
- Trimming the rules file (from 250 to 170 lines) measurably improved task success.

Inherited from industry research, to be validated within each project:

- Security gates, after an industry finding that roughly 45% of AI-generated code carried security issues.
- Default-deny on external surfaces, after a real third-party platform vulnerability (Lovable CVE, 2025).
- No destructive git or database operations, after a well-known production-database deletion incident (Replit, 2025).
- Context discipline, after research showing a 19% slowdown when context is overloaded on mature codebases.

By the numbers: 9 milestones, 301 tests, 64 reusable lessons, 4 retrospectives. Now broadened by a cross-project harvest of six more projects, ratified by a 13-seat blind council. Version history: v1.1 to v4.2.

What is inside the package

A reference map of all 121 files, grouped by area. During a live demo, this is what to say when you open any file.

Root - control and entry files

The brain of the package: rules, permissions, the full design, and the build commands.

START_HERE.md	A five-minute orientation for any new team member or agent: the mental model, the first question to ask, and what is measured versus inherited. The natural starting point for a walkthrough.
Project_Implementation_Prompt.md	The owner's paste-ready kickoff prompts for a fresh agent - a new-project variant and a mid-project variant, each ending in a comprehension echo-back the owner checks before any work begins.
AGENTS.md	The house rules every coding agent follows in this repo (kept to about 80 lines on purpose, because overloaded context lowers success). A project-specific section plus a universal section that cannot change without a logged decision.
CLAUDE.md (symlink)	An alias pointing to AGENTS.md so the Claude tooling finds it by its expected name. One file, two names - same rules.
permission-matrix.md	The default-deny permission rules: what the agent may not do, the blocking-severity taxonomy, and guardrails for driving a live UI. A claimed pass without file-and-line evidence is automatically treated as blocking.
pipeline-design.md	The full specification (about 900 lines): the two layers, the five stages, issue management, automation hooks, memory, and anti-patterns. A reference document, not a linear read.
pipeline-schema.html	A one-page visual of the whole flow. The 'show the picture' moment in a demo. Present in all versions.
GP-v2.2-presentation.html	A 16-slide introduction deck that opens in a browser, for narrating the overview.
README.md	The package's front door: fill in the placeholders, run make check, and follow the read order.
Makefile	The single source of truth for build, test and lint commands (make check, make test, make bootstrap-check). Agents use these instead of ad-hoc shell commands.
pyproject.toml	The Python project and dependency definition (ships with placeholders). Rule: every import must be matched by a declared dependency.
note.txt	A quick status card: project, latest milestone, last session, what is done and what remains. Refreshed at the end of every session.
.gitignore	Paths Git should ignore (virtual envs, caches, the personal environment file).
.gitleaks.toml	The secret-scanning configuration, used by the build, by CI, and by the pre-commit hook so credentials never reach the repo.
.mcp.json	Configuration for external tool servers (for example the GitHub integration) the agent connects to.
.pre-commit-config.yaml	Opt-in pre-commit checks (formatting, secret scan, hygiene) that back up CI on the developer's machine.

.agents/rules/ - the canonical rulebook

The engineering rules the agent reads at the start of every session.

README.md	'Read every file in this directory at the start of every session - these are the rules.' The entry note to the rulebook.
practices.md	Portable engineering rules: the frequently used subset of the lessons, applied across machines and developers.
playbook-seeds.md	An append-only collection of generalizable lessons ('seeds'), including the v2.2 and v3 ratified blocks plus a clearly-marked Agent-Native candidate sub-block. Never edited, only added to.
environment.md.template	A template for each developer's own machine settings (shell, ports). Renaming it drops it from version control. Removes 'works on my machine' drift.

.claude/ - agent harness and commands

Settings and ready-made commands that automate the agent's behavior. (Some are created once by hand from the docs/guides.)

settings.json	The team's shared harness configuration: permissions plus two baseline safety hooks (block writes to secret files; run make check after edits).
skills/file-issue	Opens a well-formed issue for a problem found (a failing check, a flaky test, a dependency alert).
skills/triage-issue	Reads a single issue, reproduces it if possible, labels it by real root cause, and posts a diagnosis.
skills/fix-issue-prepare	Creates a branch and writes a failing test for a triaged issue, then stops for review before any fix.
skills/fix-issue-implementation	On a prepared branch, makes the smallest fix to turn the test green, then runs lint, tests and the commit gate.
skills/log-decision	Appends a new decision record to the decisions log in the standard format.
skills/standup	Prints a fast, no-AI snapshot of project state; it simply runs the standup script.
skills/test-and-commit	Runs the test suite; if it passes, drafts a commit message and asks before committing.
skills/repo-review	Reviews the current branch against this repo's documented practices, not generic advice.
skills/retrospect	Runs the structured retrospective at milestone closure to measure which disciplines actually helped.
skills/quarterly-handover	Every third milestone, generates a full state dump so a fresh session can pick the project up cleanly.

.github/ - continuous integration and team boundaries

Automated checks and cross-team file ownership.

CODEOWNERS	Marks the build and deployment files DevOps owns as a cross-team boundary; changes to them require DevOps review.
workflows/ci.yml	Pure CI: the lint, test and secret-scan gate, hardened with least-privilege tokens, timeouts and pinned action versions.
workflows/issue-agent.yml	A CI-triggered agent that ships in 'shadow mode' for the first milestone (it observes but does not act automatically).

docs/ - project documentation

Requirements, architecture, decisions, the session log, templates, and this overview.

architecture.md	The system's shape and its contract with neighboring systems; treated as an adversarial cross-check against the requirements.
prd.md	Product requirements with stable IDs. Tests and pull requests cite those IDs permanently for traceability.
decisions.md	A lightweight log of architectural decisions, pre-seeded with universal ones; each project numbers its own from a reserved range to avoid collisions.
process-log.md	An append-only session journal; each entry ends with a 'Lesson' tag that feeds the reusable-rules collection.
closure-checklist.md	The milestone-closure checklist: the bootstrap gate, the per-wave Code+Tester review, the blocking closure security review (before deploy), the quality gate, and deploy / go-live verification.
security-baseline.md	The web/API security baseline (v3): no plaintext credentials or default-admin password (an enforced gate), server-side authorization on mutating routes, a CORS allowlist, startup config validation, and encryption of credentials/PII at rest.
onboarding.md	A day-one to week-one guide for a new engineer or agent (about 40 lines).
tool-suitability.md	A one-page filter for which tasks suit an AI agent (strong / medium / weak fit) versus which need a human.
feature-catalog.md	The single canonical source for the customer-facing feature set, tied to requirement IDs.
deliverables-plan.md	What the project produces, for whom, and when. Filled in at bootstrap and updated at each closure.
codex-audit.md	If inherited code came with the project, a step to separate design from code before building on it; skipped for greenfield projects.
claude-harness-config.md	Instructions to manually create the two harness files the generator cannot write directly.
claude-skills-content.md	The content for the ten starter commands, to be created under the commands directory after cloning.
HANDOVER-v2.1-material.md	A provenance record of the raw material that produced v2.1.

HANDOVER-v2.2-material.md	A provenance record of the material and ratification that produced v2.2, so the cut can be audited later.
HANDOVER-v3-material.md	A provenance record of the six source projects and the 13-seat blind council that produced v3, including the intra-ecosystem-independence caveat and a note for the next maintainer.
HANDOVER-v3.1-material.md	A provenance record of the v3.1 increment: the first field run of v3 (all six adopt validations confirmed), the 11-seat council, and the caveats for the next maintainer (self-validation cap, gateway-family evidence).
HANDOVER-v3.2-material.md	A provenance record of the v3.2 increment: the external research corpus, the two-phase council, and the owner's decision to hold the autonomy ladder as a design target rather than an operating mode.
HANDOVER-v3.3-material.md	A provenance record of the v3.3 increment: the owner's milestone-cadence review directive, the chair-delegated council, the provisional A0.5 mode with its automatic fallback, and the closure of the field project's harvest.
wave-checklist.template.md	The v3.1 wave-close checklist: filled and committed at every wave close, each tick citing fresh evidence from the wave's commit range, with a ledger row for skipped or waived checks and the risk-tier review rule.
fixpack.template.md	The v3.4 fix package - the deploy gate for post-production bug fixes: per-fix evidence, security floor, full regression on the bundle, and the owner's out-of-sandbox verification signature. Its lesson lines feed the project experience record automatically.
autonomy-protocol.md	The long-term autonomy north star (A0/A1/A2) - a recorded design target, deliberately NOT active: today the owner reviews every wave and milestone; the document exists so the destination is engineered before it is ever enabled.
closure-report.template.md	The v3.2 owner review pack: a two-page, evidence-linked report generated at every milestone closure - what shipped, the git record, trust telemetry, all ledgers, and a plain-prose explanation of what changed structurally.
EXPERIENCE.template.md	The v3.1 living experience document: appended at every milestone closure and required to be fresh before any quarterly handover, so organizational harvesting becomes collection instead of a campaign.
license-review.template.md	A template to review the license and commercial-use terms of any third-party engine the project wraps or forks, completed before building (a bootstrap gate).
pm-status.template.md	A template for a manager-readable status snapshot in plain language: every line states what is there and what is missing.
project-brief.template.md	A short brief filled in before starting a project; it saves roughly 30-40 minutes of back-and-forth at bootstrap.
executive-overview.md	This manager-facing overview in editable Markdown - the single source of truth for the document, refreshed at each version cut.
executive-overview.pdf	The polished, hand-to-the-manager rendering of the overview (this document).
executive-overview.gen.py	The generator that renders both the .md and .pdf from one content source; re-run it after editing to refresh both.

docs/external-skills/ - reference only

Reference copies captured from an open-source toolkit. Not activated; kept as design source material.

README.md	Why these reference copies are here, with a clear 'do not activate' note.
writing-plans.md	Reference copy of a plan-writing command.
requesting-code-review.md	Reference copy of a code-review-request command.
subagent-driven-development.md	Reference copy of a subagent-driven-development command.
using-git-worktrees.md	Reference copy of a git-worktrees command.

docs/handovers, plans, retrospectives, reviews

Folders where working outputs are written.

handovers/handover_q1.txt	A quarterly handover template (milestones 1 to 3), produced by the quarterly-handover command.
plans/.gitkeep	Keeps the empty 'plans' folder in version control; milestone plans are written here.
retrospectives/.gitkeep	Keeps the empty 'retrospectives' folder; retrospectives are written here.
reviews/.gitkeep	Keeps the empty 'reviews' folder; code and security review outputs are written here.

scripts/ - deterministic state tools

Fast, cheap, no-AI operations for reading project state.

README.md	A table of the scripts and what each one does.
standup.sh	Prints project state in about five seconds: latest log, open decisions, latest plan / review / handover, and rules-file size.
bootstrap-check.sh	An executable bootstrap gate: it checks for leftover placeholders, a version-stamped health endpoint, filled-in core docs, required decisions, the license review, and (in v3) any default-admin password or plaintext credential in the source. It enforces discipline by machine, not by memo.

src/app/ - a minimal working skeleton

The smallest runnable app so the very first commit is already green and testable.

adapter/main.py	A minimal web app exposing a version-stamped /health endpoint, so 'which build is live?' is answerable with a single command.

clients/___init___.py	A typed boundary for all external API calls, with a fake implementation kept alongside for tests.
workers / workflows (___init___.py)	Empty package skeletons for background workers and workflows, to be filled in by the project.
package markers (___init___.py)	Empty files that turn the source folders into importable Python packages.

subagent-profiles/ - mandatory reviewers

The independent reviewers with fresh eyes. In v3, a Code Reviewer and a Tester run at the end of every wave; the Security Reviewer runs once at milestone closure, blocking before any deploy.

README.md	When each profile fires, how it is invoked, and whether a plan may override it - a short summary table (updated for the v3 review-loop restructure).
Code-Reviewer.md	The mandatory code-review persona: a senior engineer reviewing the wave's code with fresh eyes (never their own).
Tester.md	The mandatory per-wave tester persona (v3): fresh eyes that prove the wave against its acceptance criteria, writing or extending tests red-to-green so every criterion has a passing citing test.
Security-Reviewer.md	The mandatory security-review persona; in v3 it runs at milestone closure over the whole milestone's surface and is blocking before any deploy.

tests/ - the day-one green baseline

Tests that guarantee a fresh clone passes its checks immediately.

unit/test_health.py	The day-one baseline test plus the version-stamp contract for the health endpoint; any future failure is a real regression.
package markers (___init___.py)	Empty files that make the test folders importable packages.

Anticipated questions

What does this give us?

A repeatable engineering pipeline. Parallel agent work gave a 4-6x speed-up in the source project, mandatory independent review caught real defects at every milestone, and each project accumulates lessons that make the next one faster.

How do we control the risk?

A default-deny permission system plus hard blocks on the most dangerous actions (no destructive git or database operations, no committing secrets), and a rule that any claimed 'pass' without evidence is treated as a failure. These guardrails come from real industry incidents.

How long does it take to set up?

About 4 to 8 hours of one-time bootstrap per project. A filled-in project brief cuts roughly 30-40 minutes off that, and an executable check confirms the setup is complete.

Are we just trusting the AI?

No. A human approves the milestone plan, a human merges every pull request, and high-risk areas (authentication, personal data, payments, migrations) require senior human review. The agent proposes; people approve.

How solid is the evidence?

We separate it honestly: some parts were measured directly in the source project and can be trusted; others are inherited from industry research and are validated within each new project. The overview lists which is which.